# Design Patterns în Java — Teorie

Modul construit lecție cu lecție · corespondent exercițiilor din repo

## Lecția 0 — Ce este un design pattern (puntea de la interfețe)

### De unde venim

În `java-abstractizare` ai scris interfața `Duplicabil`, cu o singură metodă: `Duplicabil interfaceDuplicate();`. Au semnat-o trei ierarhii care n-au nicio rudă comună: figurile din `formeGeometrice`, intervalele din `OrarScheduler` (`Pauza`, `Examen`) și `Orar` însuși. Codul care copiază nu întreabă CE e obiectul, ci doar dacă ȘTIE să se copieze.

Tot acolo ai pus teza: **`abstract` răspunde la „ce ești”, `interface` la „ce poți face”**. Și ai folosit un contract scris de altcineva: `Comparable`. Clasa ta implementează `compareTo`, iar `Collections.sort` o sortează fără să știe nimic despre ea, doar că respectă contractul.

Ține minte propoziția asta, pentru că e singura idee din tot modulul:

> **Depinzi de contract, nu de clasă.**

Toate pattern-urile pe care le facem sunt aceeași idee, aplicată la trei întrebări diferite.

### Ce e, de fapt, un pattern

Un **design pattern** nu e o bibliotecă, o clasă gata făcută sau o sintaxă nouă. E un **nume pentru o soluție care se repetă** la o problemă care se repetă. Cineva a lovit aceeași problemă de o mie de ori, a găsit forma care merge și i-a dat un nume, ca să putem vorbi despre ea în două cuvinte în loc de zece minute.

Beneficiul e dublu:

- **Nu reinventezi** — când recunoști problema, știi deja forma soluției.
- **Comunici scurt** — spui „aici e un Strategy” și toată echipa înțelege structura, fără să citească fiecare linie.

Riscul, pe care îl vom evita: pattern-ul aplicat unde nu trebuie. Un pattern rezolvă o durere concretă. **Fără durere, fără pattern** — altfel adaugi complexitate degeaba.

### Firul logic al modulului — trei întrebări

| Grup | Întrebarea la care răspunde | Pattern-uri |
|---|---|---|
| **Comportament** | *cine se comportă și cum?* | Strategy, Observer, State |
| **Structură** | *cum compun obiectele?* | Decorator, Adapter |
| **Creare** | *cine și cum creează obiectele?* | Factory Method, Builder, Singleton |

Modulul le ia pe rând. Ordinea nu e întâmplătoare: începem cu **comportamentul**, pentru că e cel mai aproape de interfețe. Primul pattern — Strategy — e practic un pas mic peste ce știi deja.

## Lecția 1 — Strategy

> **Un cuvânt:** comportament interschimbabil. Închizi fiecare variantă de „cum se face ceva” într-o clasă separată care semnează același contract și o schimbi când vrei — chiar la runtime.

### Problema (exercițiul `ex1`: costul de livrare)

Un magazin calculează costul de livrare al unei comenzi. Costul depinde de metodă: standard, express, promoție gratuită. Prima variantă la care se gândește oricine:

```
public enum TipLivrare { STANDARD, EXPRESS, GRATUITA }

public double calculeazaCost(TipLivrare tip, double greutate,
                             double distanta) {
    switch (tip) {
        case STANDARD:
            return 5 + 0.5 * greutate + 0.1 * distanta;
        case EXPRESS:
            return 12 + 1.0 * greutate + 0.25 * distanta;
        case GRATUITA:
            return 0;
        default:
            throw new IllegalArgumentException("Unknown delivery type");
    }
}
```

Merge. Și e o capcană. Întreabă-te ce faci când firma adaugă `LivrareInternationala`:

- deschizi metoda asta și adaugi un `case`;
- probabil mai există un `switch` la fel în alt loc (afișare, facturare) — îl deschizi și pe ăla;
- fiecare `switch` uitat = un bug tăcut.

Aici intră o regulă nouă, **open/closed**: *o clasă ar trebui să fie deschisă la extindere, dar închisă la modificare*. Adică adaugi comportament nou scriind cod nou, nu editând cod care deja merge. Ai simțit-o deja în fișa de interfețe: un `Romb` cu arie, adăugat fără să atingi codul care adună ariile. Cu `switch`-ul de mai sus, ca să extinzi (metodă nouă de livrare) ești **obligat să modifici**. `switch`-ul pe tip e mirosul clasic care strigă „aici lipsește un pattern”.

### Ideea

Întoarce lucrurile pe dos. În loc ca o metodă să știe TOATE variantele și să aleagă cu `switch`, **fiecare variantă devine o clasă** care știe DOAR de ea și semnează un contract comun:

```
public interface LivrareStrategie {
    String getNume();
    double calculeazaCost(double greutateKg, double distantaKm);
}
```

`Comanda` nu mai conține nicio formulă și niciun `switch`. Ține o `LivrareStrategie` și o **întreabă**:

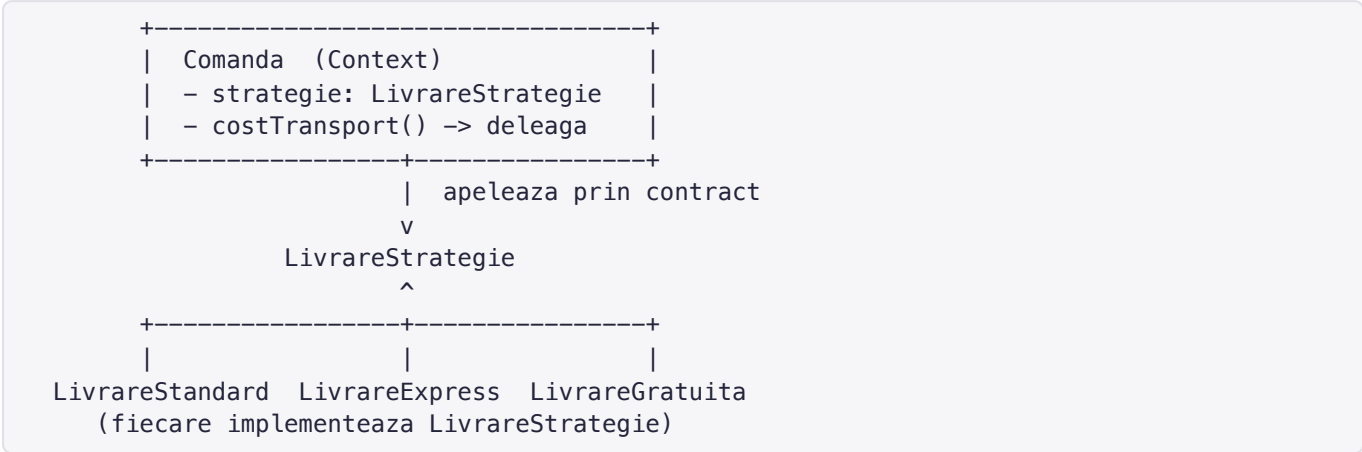
```
public double costTransport() {
    return strategie.calculeazaCost(greutateKg, distantaKm);
}
```

O metodă nouă de livrare = **o clasă nouă**, atât. Nu atingi `Comanda`, nu atingi celelalte strategii. Închis la modificare, deschis la extindere — de data asta chiar.

### Cele trei roluri

| Rol | În exercițiu | Ce face |
|---|---|---|
| **Strategy** (contractul) | `LivrareStrategie` | declară operația interschimbabilă |

| | | |
|---|---|---|
| **ConcreteStrategy** | `LivrareStandard`, `LivrareExpress`, `LivrareGratuita` | fiecare, o singură variantă a algoritmului |
| **Context** | `Comanda` | ține o strategie, deleagă spre ea, o poate schimba |




### Mecanismul de dedesubt

Nu e magie — sunt trei lucruri pe care le știi deja, puse împreună:

1. **Polimorfism prin contract.** `strategie.calculeazaCost(...)` cheamă implementarea clasei reale din spatele interfeței. Exact ce făcea `interfaceDuplicate()` chemat pe un `Duplicabil`: se copia o figură sau un `Examen`, după obiectul real.
2. **Delegare.** `Comanda` nu *face* calculul, îl *pasează* mai departe. Contextul e un dispecer, nu un executant.
3. **Injectare.** Strategia intră din afară (prin constructor sau `schimbaStrategie`), nu e creată înăuntru. De-aia o poți schimba la runtime — și de-aia contextul nu depinde de nicio clasă concretă.

Detaliul care le leagă: `calculeazaCost` primește greutatea și distanța ca **parametri**, nu le ține strategia în câmpuri. Așa aceeași instanță `LivrareStandard` servește orice comandă — strategia e *fără stare*, deci refolosibilă și fără surprize. (Vezi întrebarea din bonusul exercițiului.)

### Când îl folosești

- Ai **mai multe variante** ale aceluiași lucru (calcul, sortare, validare, format de export) și vrei să comuți între ele.
- Vezi un **`switch`/lanț de `if`** pe un „tip” care se tot repetă prin cod.
- Vrei să **adaugi variante** fără să atingi codul existent.
- Vrei să poți schimba comportamentul **la runtime**, nu doar la compilare.

### Când NU îl folosești

- Ai **o singură variantă** și nu se întrevăd altele. Un `if` simplu e mai onest decât trei clase și o interfață.
- Variantele diferă printr-o **singură valoare**, nu printr-un algoritm (atunci e un parametru, nu o strategie).
- Pattern-ul aplicat de dragul pattern-ului adaugă doar ceremonie. Amintește-ți: fără durere, fără pattern.

### Capcane frecvente

- **Context cu `switch` rămas.** Dacă `Comanda` tot decide cu `if` care strategie să folosească, n-ai mutat problema — ai dublat-o. De-aia constrângerea din exercițiu interzice `if` / `switch` pe tip în `Comanda`.
- **Strategie cu stare de comandă.** Dacă `LivrareStandard` ține greutatea în câmp, n-o mai poți refolosi între comenzi. Ține strategia fără stare; datele vin ca parametri.
- **Prea multe strategii minuscule.** Dacă variantele diferă cu o linie, poate era destul o lambda sau un parametru.

### Legătura cu ce urmează

În bonusul exercițiului scrii `LivrareCuReducere`: o strategie care primește **altă strategie** în constructor și îi ajustează rezultatul. O strategie care împachetează o strategie — asta e deja un **Decorator** (lecția 4). Strategy ți l-a arătat gratis; acolo doar îi punem numele.